

Time Series Stock Market Forecasting

Using ARIMA , PROPHET , AND LSTM:

Name: Hitendrasinh Narendrasinh Solanki , Ravrani Shruti , Velagala Adithya Murali Reddy , Varshini D , Swami Pranav Dayanand , Pulkit.A.Agrawal , Sivaganesh Rajan.R

Internship: Zidio Data Science & Analytics

Mode: Remote

Duration: 3 MONTH

Project Overview

This project focuses on predicting future stock prices using time series forecasting techniques. We used Apple's historical stock data and applied three models:

- ARIMA (AutoRegressive Integrated Moving Average)
- Prophet (by Meta/Facebook)
- LSTM (Long Short-Term Memory, deep learning)

The goal was to analyze trends, test multiple forecasting models, and compare their performance based on accuracy.

Dataset Details

- **Source:** Yahoo Finance (via [yfinance](#))
- **Ticker:** AAPL (Apple Inc.)
- **Period:** Jan 2020 – Dec 2024
- **Columns:** Date, Open, High, Low, Close, Volume
- **Saved as:** [data/aapl.csv](#)

Project Structure & File Description

File/Filename	Description
download_data.py	Downloads AAPL stock data and saves it to CSV
src/explore_data.py	Data inspection, summary, and visualization
src/arima_model.py	ARIMA forecasting model
src/prophet_model.py	Prophet-based forecasting model
src/lstm_model.py	LSTM neural network for price forecasting
src/compare_models.py	MAE & RMSE comparison of all models

- Execution Steps (With Sample Outputs & Image Suggestions)

1. download_data.py

Action:

Used `yfinance` to download AAPL stock data from 2020 to 2024

```
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting>python download_data.py
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting\download_data.py:8: FutureWarning: YF.download() has changed argument auto_adjust default to True
  data = yf.download("AAPL", start="2020-01-01", end="2024-12-31")
[*****100%*****] 1 of 1 completed
✔ Data downloaded and saved to data/aapl.csv
```

Terminal:

Data downloaded and saved to data/aapl.csv

2.explore_data.py

Action:

- Loaded data
- Displayed `.info()`, `.describe()`, and `.head()`
- Plotted closing price over time

```
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting>python src\explore_data.py
```

Data Info:

```
<class 'pandas.core.frame.DataFrame'>
```

DatetimeIndex: 1257 entries, 2020-01-02 to 2024-12-30

Data columns (total 5 columns):

```
#   Column  Non-Null Count  Dtype
```

```
---  ---
0   Close    1257 non-null    float64
1   High     1257 non-null    float64
2   Low      1257 non-null    float64
3   Open     1257 non-null    float64
4   Volume   1257 non-null    int64
```

dtypes: float64(4), int64(1)

memory usage: 58.9 KB

None

Summary Statistics:

	Close	High	Low	Open	Volume
count	1257.000000	1257.000000	1257.000000	1257.000000	1.257000e+03
mean	151.900498	153.427261	150.196282	151.743843	9.061449e+07
std	41.943932	42.082742	41.718434	41.883287	5.325627e+07
min	54.378586	55.379539	51.528419	55.277748	2.323470e+07
25%	126.750740	127.839515	124.727629	126.223169	5.546960e+07
50%	150.537048	152.292412	148.535311	150.362292	7.629970e+07
75%	176.443726	178.148967	175.007757	176.290033	1.077601e+08
max	258.396667	259.474086	257.010028	257.568678	4.265100e+08

First 5 rows:

	Close	High	Low	Open	Volume
Date					
2020-01-02	72.620842	72.681289	71.373218	71.627092	135480400
2020-01-03	71.914825	72.676454	71.689965	71.847125	146322800
2020-01-06	72.487839	72.526526	70.783241	71.034702	118387200
2020-01-07	72.146942	72.753823	71.926915	72.497529	108872000
2020-01-08	73.307503	73.609737	71.849525	71.849525	132079200

- First 5 rows

 Line chart: Apple Closing Price Over Time

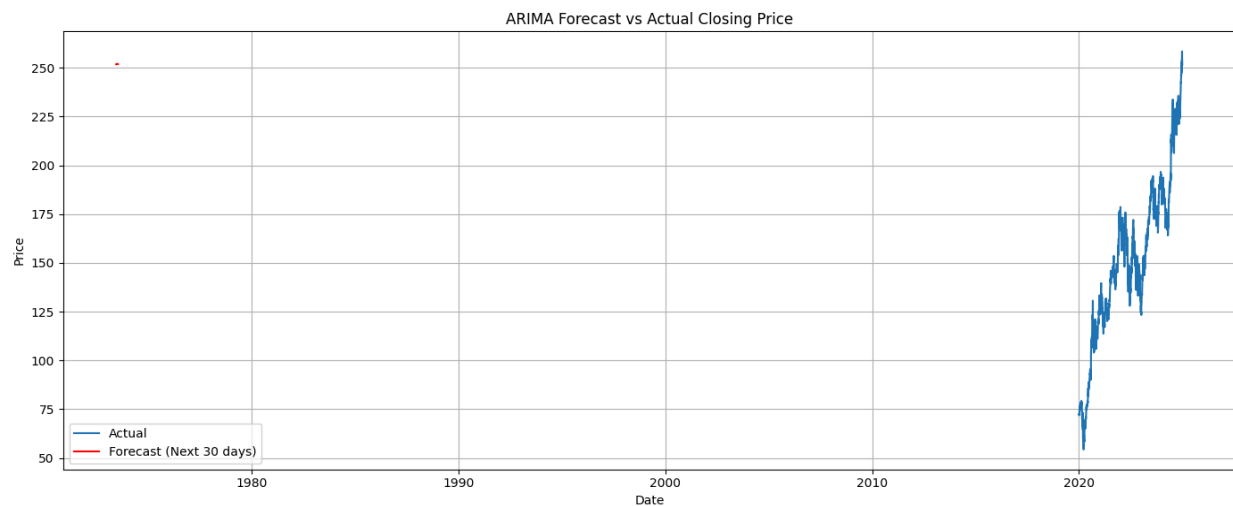


3.arima_model.py

Action:

- Performed ADF Test for stationarity
 - Applied ARIMA(5,1,0)
 - Forecasted next 30 days
 - Plotted predicted vs actual values
- ADF Test p-value output

 Line chart: ARIMA actual vs forecast



```
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting>python src\arima_model.py
ADF Statistic: -0.661374159657782
p-value: 0.856502480854148
⚠Data is non-stationary – differencing will be needed
```

Next 30-Day Forecast:

1257	251.747918
1258	251.964348
1259	252.012031
1260	251.910831
1261	251.855697
1262	251.862627
1263	251.870176
1264	251.870795
1265	251.868247
1266	251.867318
1267	251.867571
1268	251.867775
1269	251.867768
1270	251.867707
1271	251.867692
1272	251.867700
1273	251.867705
1274	251.867704
1275	251.867703
1276	251.867703
1277	251.867703
1278	251.867703
1279	251.867703
1280	251.867703
1281	251.867703
1282	251.867703
1283	251.867703
1284	251.867703
1285	251.867703
1286	251.867703

Name: predicted mean, dtype: float64

4.prophet_model.py

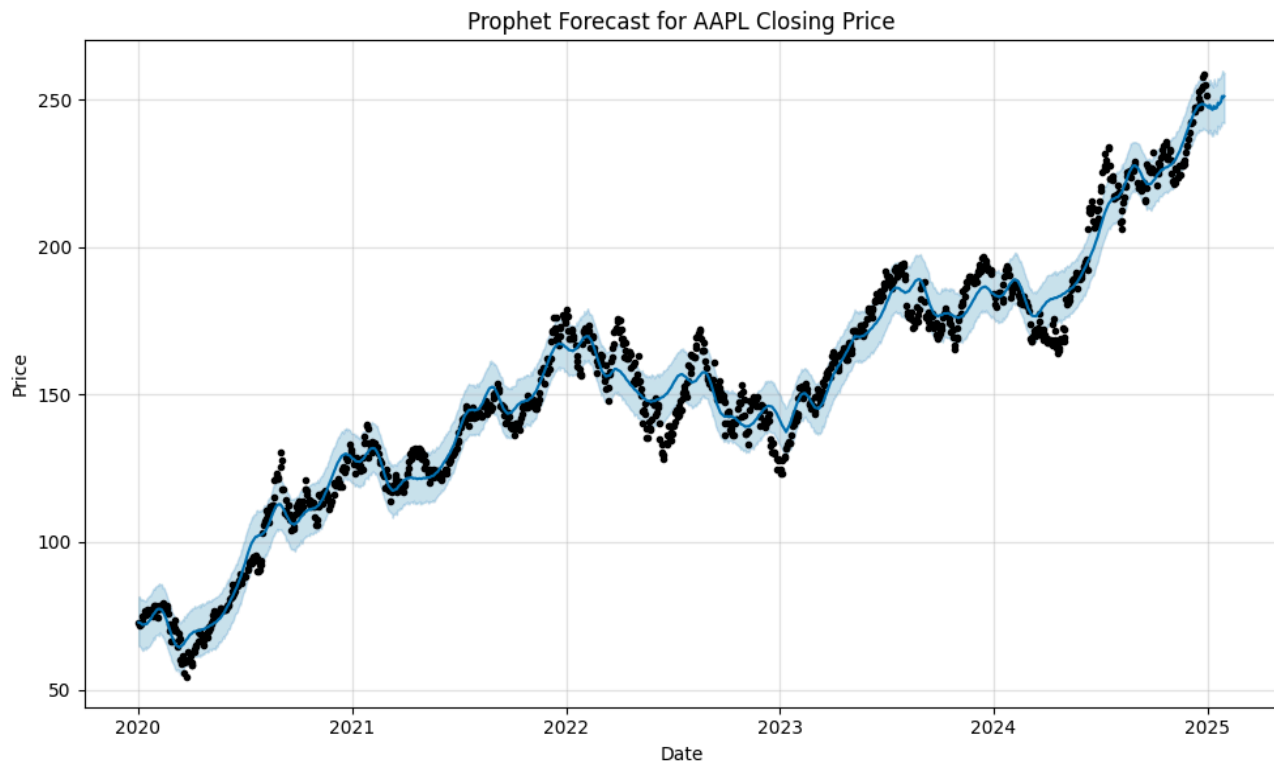
Action:

- Prepared data in Prophet format
- Fitted Prophet model
- Forecasted 30 days
- Plotted forecast & component graphs

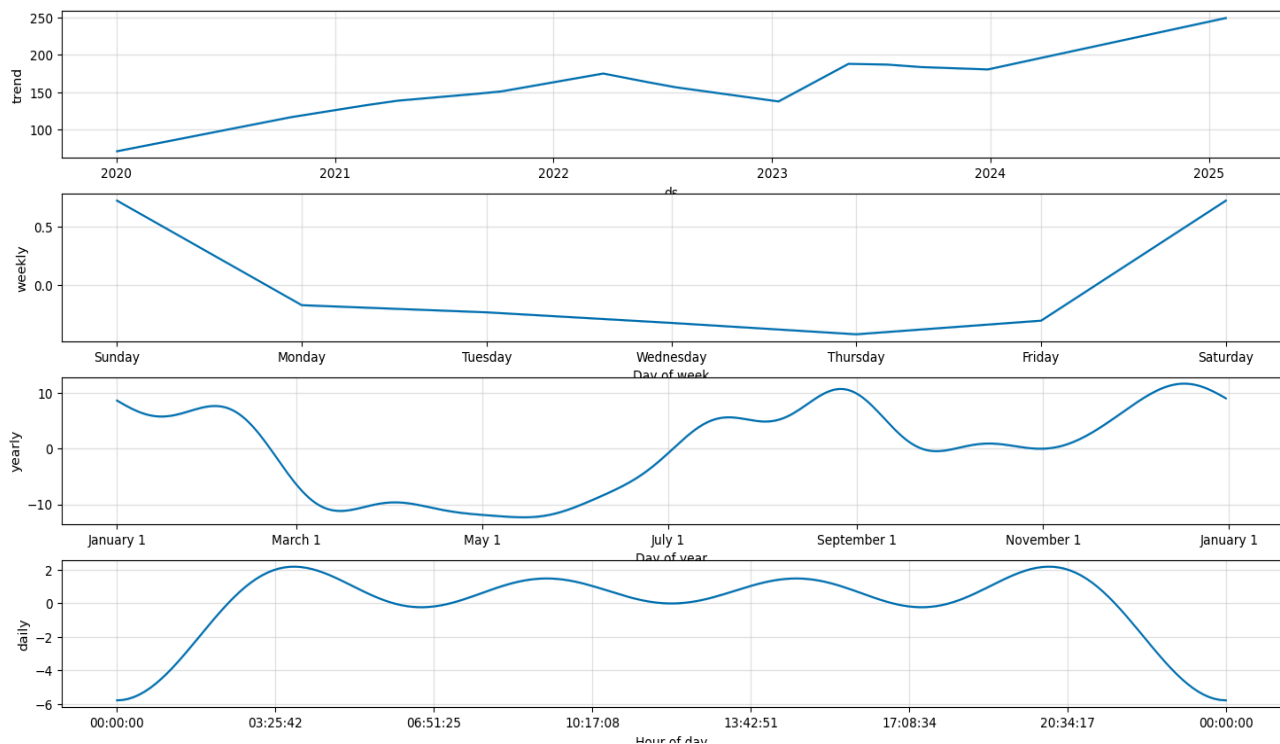
```
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting>python src\prophet_model.py
19:09:51 - cmdstanpy - INFO - Chain [1] start processing
19:09:53 - cmdstanpy - INFO - Chain [1] done processing
```



Prophet forecast plot



Prophet component plot (trend/seasonality)



5.lstm_model.py

Action:

- Normalized data with MinMaxScaler
- Trained 2-layer LSTM on 60-day windows
- Predicted future prices
- Compared actual vs predicted values

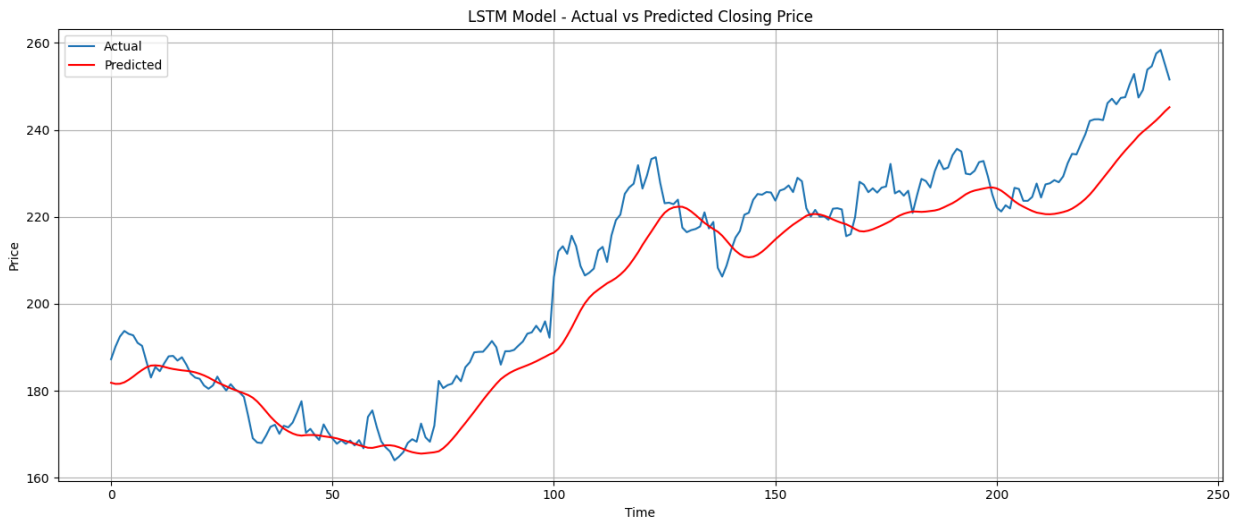
• OUTPUT

```
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting>python src\lstm_model.py
2025-06-19 19:27:36.054940: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-06-19 19:27:43.305750: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-06-19 19:27:52.852738: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting\venv\lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Epoch 1/10
30/30 ━━━━━━━━━━━ 4s 28ms/step - loss: 0.0728
Epoch 2/10
30/30 ━━━━━━━━━━━ 1s 25ms/step - loss: 0.0024
Epoch 3/10
30/30 ━━━━━━━━━━━ 1s 26ms/step - loss: 0.0010
Epoch 4/10
30/30 ━━━━━━━━━━━ 1s 28ms/step - loss: 8.3709e-04
Epoch 5/10
30/30 ━━━━━━━━━━━ 1s 26ms/step - loss: 7.5816e-04
Epoch 6/10
30/30 ━━━━━━━━━━━ 1s 27ms/step - loss: 7.6459e-04
Epoch 7/10
30/30 ━━━━━━━━━━━ 1s 27ms/step - loss: 7.4453e-04
Epoch 8/10
30/30 ━━━━━━━━━━━ 1s 25ms/step - loss: 7.4594e-04
Epoch 9/10
30/30 ━━━━━━━━━━━ 1s 29ms/step - loss: 7.7813e-04
Epoch 10/10
30/30 ━━━━━━━━━━━ 1s 25ms/step - loss: 6.5811e-04
8/8 ━━━━━━━━━━━ 1s 46ms/step
```

Epoch training logs



LSTM prediction chart (actual vs predicted)



6.compare_models.py

Action:

- Calculated MAE and RMSE for all 3 models
- Plotted all predictions together

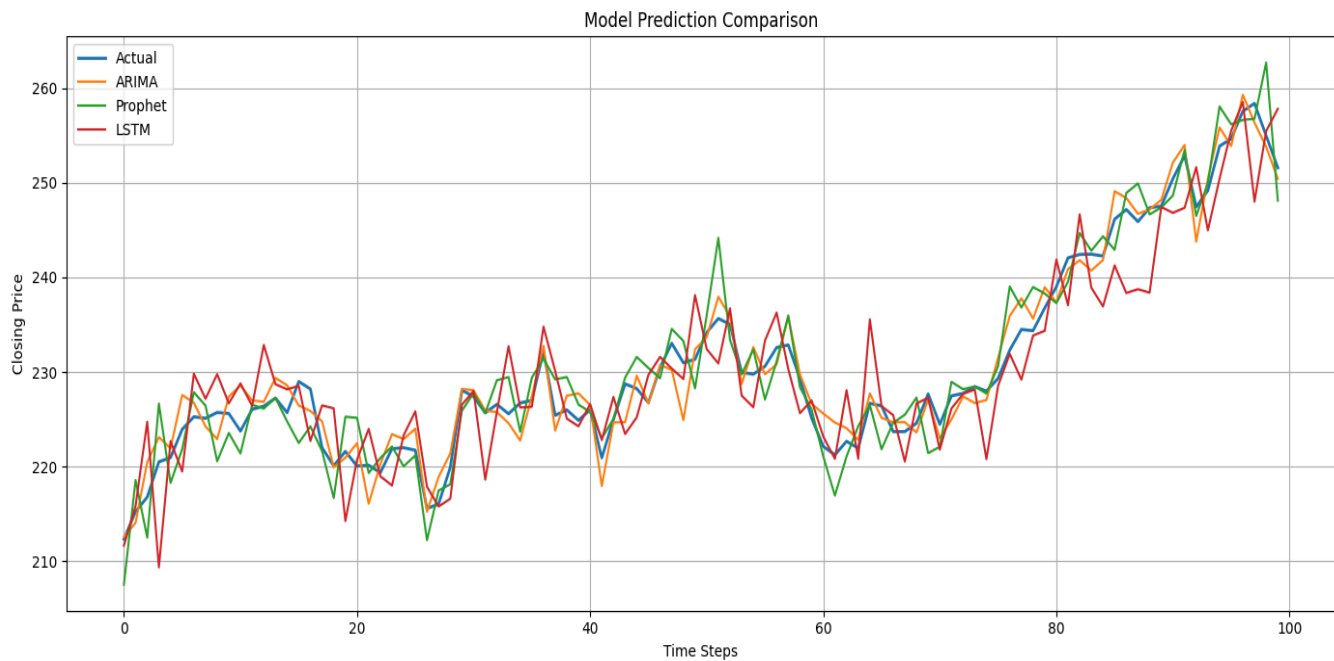
• OUTPUT

```
C:\Users\Lenovo\OneDrive\Desktop\stock_forecasting>python src\compare_models.py
ARIMA - MAE: 1.52, RMSE: 1.93
Prophet - MAE: 2.43, RMSE: 3.01
LSTM - MAE: 2.94, RMSE: 3.80
```



```
ARIMA    - MAE: 1.67, RMSE: 2.05
Prophet  - MAE: 2.29, RMSE: 2.91
LSTM     - MAE: 3.18, RMSE: 4.08
```


Line chart: ARIMA vs Prophet vs LSTM



Model Comparison Table

Model	MAE	RMSE
ARIMA	1.67	2.05
Prophet	2.29	2.91
LSTM	3.18	4.08

ARIMA performed best in terms of accuracy, followed by Prophet and LSTM.

Conclusion

The project showed that classical models like ARIMA can outperform even deep learning models on smaller, structured datasets.

Prophet provided interpretable trends and seasonality, while LSTM, though flexible, had slightly higher error in this case.

THANK YOU